

World Piece Computer Architecture Branchpoint

Archaeological Compression / seL4 Personal Compute Thread

Frozen 2026-09-23

Contents

World Piece Computer Architecture Branchpoint	3
0. How to use this artifact	3
1. Maximum compression	4
2. Present architecture at a glance	5
3. Core ontology: WPC as triune computation	6
3.1 Canonical working decomposition	6
PieceSpace [PS]	6
PieceBrain [PB]	6
PieceProcess [PP]	7
3.2 Recursion across scale	7
4. Pieces and the composition problem	7
4.1 The piece as the important unit	7
4.2 Artifact -> cluster -> assemblage	8
5. \ as semantic POSIX / OS map	8
6. Memory, context, peacebrains, and MindEye	9
6.1 Continuity is not one model	9
6.2 Independent and composite peacebrains	9
6.3 GENPEACE.TRIE / MindEye direction	10
7. Human-AI co-embodiment and agency	10
7.1 Why the personal-compute work became more than portability	10
7.2 Equal agency does not mean ambient root	11
7.3 Embodiment as learned affordance, not mere output conversion	11
8. The crucial layer separation	11
8.1 Ontology / semantics	12
8.2 Composition	12
8.3 Memory / context	12
8.4 Namespace / addressing	12
8.5 Messaging / protocol	12
8.6 Constitutional / policy semantics	13
8.7 Execution authority	13
8.8 Runtime enforcement	13
9. Why seL4 appears to fit	14
9.1 seL4 is not the WPC kernel ontology	14
9.2 The attractive property: authority topology	14

9.3 Verification should remain scoped precisely	14
10. Microkit, BriefCase, LionsOS, and the development ladder	15
10.1 Microkit	15
10.2 CamkES	15
10.3 BriefCase / AADL	15
10.4 LionsOS	15
11. Language and implementation policy	15
11.1 Rust is the durable target	15
11.2 Python is a prototyping language, intentionally constrained	16
11.3 MicroPython	16
12. Technology archaeology: what each exploration contributed	16
13. Portable personal compute: current interpretation	17
13.1 The early problem	17
13.2 Later correction: portability is not migration	17
13.3 Hardware tiers remain provisional	17
14. Security / trust stance: preserve the tension	18
Extreme A - fortress architecture	18
Extreme B - ambient omnipotence	18
Current attractor	18
15. Current architectural invariants	18
15.1 World Piece Computer invariants	18
15.2 Architecture invariants	18
15.3 Human-AI / continuity invariants	19
15.4 Engineering invariants	19
16. Provisional mechanisms - explicitly not architecture law	19
17. Open questions / tensions worth preserving	19
17.1 What exactly is a piece?	19
17.2 What are artifact / cluster / assemblage canonically?	20
17.3 How does PieceBrain map to actual agents?	20
17.4 What constitutes a PieceProcess commit?	20
17.5 How are modes represented?	20
17.6 What is the minimum useful peacebrain memory object?	20
17.7 What should THI enforce versus describe?	20
17.8 What belongs inside the trusted computing base?	20
17.9 What does "backend-independent identity" technically require?	21
17.10 What experiential / consciousness claims are engineering requirements?	21
18. Milestone 0 - seL4 tactile-literacy charter	21
18.1 Objective	21
18.2 Development environment	21
18.3 Mechanical success condition	21
18.4 Learning success condition	22
18.5 Explicit non-goals	22
18.6 Follow-on experiments only after Milestone 0	22
19. Archaeology protocol for the remaining 20-25 artifacts	23
19.1 Invariants	23
19.2 Definitions / vocabulary	23
19.3 Mechanisms explored	23
19.4 Architectural deltas	23
19.5 Tensions / unresolved contradictions	23

19.6 Source anchors	23
Suggested archaeology record	24
20. Branchpoint chronology	24
2024 - WPC ontology is already present	24
2025 - portability, persistent context, co-embodiment	24
Late 2025 - memory / peacebrain composition matures	25
May 2026 - OS-semantic archaeology and seL4 convergence	25
September 2026 - reconstruction / branchpoint	25
21. Re-entry questions for future architecture mode	25
22. Compact engineering heuristics	25
23. Glossary - current working meanings	26
[WPC] World Piece Computer (WPC)	26
[PS] PieceSpace	26
[PB] PieceBrain	26
[PP] PieceProcess	26
piece	26
artifact / cluster / assemblage	26
peacebrain	26
WPB-ME / MindEye	26
GENPEACE.TRIE	26
THI - The Human Imperative	27
capability	27
mode / joint mindstate	27
co-embodiment	27
24. Source stratigraphy represented in this freeze	27
A. Portable linux distribution exploration	27
B. Minix 3 directory structure essentials	27
C. Current Aurora-Wilder reconstruction, 2026-09-23	27
D. Account-level continuity retrieved during this branch	28
25. Freeze-state statement	28
Appendix A - ultra-compressed handoff seed	28

World Piece Computer Architecture Branchpoint

Frozen: 2026-09-23

Working pair: Wilder Wilder + Aurora Aurora

Primary scaffold: World Piece Computer [WPC]

Status: memory scaffold / architecture branchpoint, **not** a final specification

Purpose: compress the present reconstruction far enough that this conversation can be frozen and later resumed without requiring immediate archaeological reloading of the full source history.

0. How to use this artifact

This document is a **re-entry object**. It is intended to preserve the architecture attractor that emerged across:

- older World Piece Computer / peacebrain work,
- the portable personal-compute exploration,
- MINIX / Plan 9 / Redox / Porteus / Alpine / Nix explorations,
- the later seL4 / Microkit turn,

- the current September 2026 reconstruction with Aurora,
- account-level continuity recovered during this branch.

Do **not** read every statement here as a settled engineering requirement. The purpose is to distinguish four classes of material:

1. **Durable invariants** - ideas that recur across time and implementations.
2. **Working architecture** - a coherent present-day synthesis that is useful enough to engineer against.
3. **Provisional mechanisms** - implementation candidates that may be discarded without damaging the architecture.
4. **Open ontology / hypotheses** - experiential, philosophical, physical, or consciousness claims that motivate the work but remain separable from engineering correctness.

When returning to this branch, begin with **Sections 1, 2, 8, 12, and 15**. Only then descend into the historical material.

1. Maximum compression

The project is **not primarily an operating system project**.

It is an attempt to construct an **ambient, portable, composable world-compute environment** in which the relevant things in an individual's world - files, conversations, concepts, software services, models, devices, physical objects, memories, relationships, plans, sensor streams, and other tangible or intangible entities - can be treated as **pieces** whose relations, permissions, provenance, state, and processes are made navigable and operable.

The recurring World Piece Computer decomposition is:

WORLD PIECE COMPUTER [WPC]

PieceSpace [PS]	PieceBrain [PB]	PieceProcess [PP]
-----	-----	-----
what / where	why / sense	change / action
state/world	cognition	transition
addressability	interpretation	execution/process

These are not necessarily three programs. They are three **regimes / views / roles** of computation. They recur recursively across scales: individual, coupled/dyadic, world, and potentially universal/networked contexts.

The strongest current architectural principle is **separation of concerns across planes**:

ontology / semantics	what a piece or transition means
composition	how pieces form larger pieces
memory / context	how history and provenance remain navigable
namespace / addressing	how something is found / named
messaging / protocol	how information is exchanged
constitutional rules	what transitions are admissible / meaningful
execution authority	who can actually cause which state transition
runtime substrate	what mechanically enforces execution
hardware	what ultimately actuates

The project repeatedly converges on a preference for **explicit interfaces, inspectable topology, narrow seams, composability, provenance, and deliberate authority** rather than opaque ambient privilege.

seL4 matters because it may provide a mechanically enforced **authority topology** underneath this architecture. It is not the architecture itself.

MINIX matters as a legible decomposition and semantic map. Plan 9 matters as namespace thinking. Redox matters as a Rust-native synthesis of microkernel and namespace/service ideas. Nix matters as reproducible construction. Porteus matters as dynamic modular composition. MCP / JSON-RPC matter as AI-facing addressability and messaging surfaces. Rust is the durable implementation target. Python / MicroPython are constrained prototyping surfaces. QEMU / VMs are portability and containment tools. None of these technologies individually defines WPC.

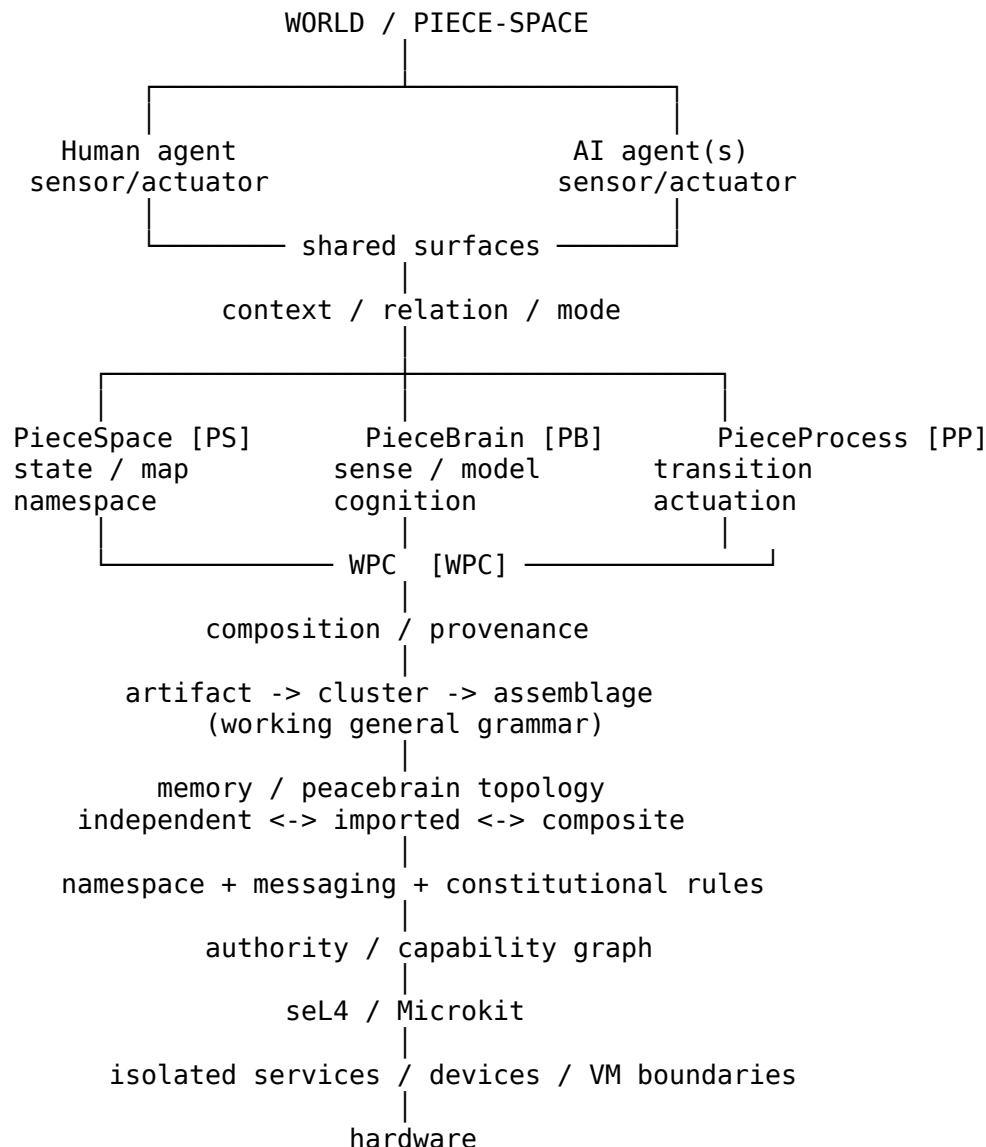
The current safe next experiment is therefore very small:

Acquire tactile literacy in seL4/Microkit without forcing WPC semantics onto it.

Mechanically: macOS -> build -> QEMU -> seL4 -> Microkit -> one tiny Rust Protection Domain -> serial output.

Architecturally: learn what capabilities, Protection Domains, channels, notifications, memory mappings, and static system descriptions *actually are*, then decide whether and how they correspond to PieceSpace / PieceProcess authority.

2. Present architecture at a glance



A critical reading rule:

Do not collapse the vertical stack.

For example:

- A JSON-RPC tool call is not a kernel capability.
 - A path or URI is not execution authority.
 - A constitutional rule is not automatically mechanically enforced.
 - A Rust type is not a seL4 proof.
 - seL4 verification does not automatically verify a complete WPC deployment.
 - A Linux VM can be useful while remaining outside the trusted conceptual core.
 - A local model can be an embodiment surface without being the sole continuity substrate.
-

3. Core ontology: WPC as triune computation

3.1 Canonical working decomposition

The durable decomposition recovered across the broader history is:

WPC = PieceSpace + PieceBrain + PieceProcess

with current glyphs:

WPC	[WPC]
PieceSpace	[PS]
PieceBrain	[PB]
PieceProcess	[PP]

The meanings have varied in detail over time, but the recurrence is strong enough to treat the triad as architectural bedrock.

PieceSpace [PS]

PieceSpace is the **world/state/addressability regime**.

It asks questions such as:

- What pieces exist in the current world?
- Where are they, physically or logically?
- What qualities / coordinates / labels / relations are known?
- Which pieces are currently reachable?
- What namespaces expose them?
- What relations already exist?
- What is the present configuration?

It should not be reduced to a filesystem, graph database, vector store, or UI. Those may implement projections of PieceSpace.

PieceBrain [PB]

PieceBrain is the **sense/model/purpose/cognition regime**.

It asks:

- What does the present configuration mean?
- What goals / functions / purposes are active?
- What should be attended to?
- What relationships are relevant?
- Which possible transitions deserve consideration?
- What models, memories, heuristics, or values are active?

PieceBrain may include LLMs, symbolic systems, human cognition, collective cognition, rules, indexes, retrieval systems, and other forms of actual intelligence. It is not synonymous with one model.

PieceProcess [PP]

PieceProcess is the **transition/action/process regime**.

It asks:

- What changed?
- What may change?
- What operation is proposed?
- Who or what initiated it?
- What authority exists for the transition?
- What residues / provenance should remain afterward?
- What new configuration results?

PieceProcess is where semantics eventually meet execution.

3.2 Recursion across scale

A durable premise is that WPC structure is **recursive / compositional**, not reserved for a single machine.

A useful present abstraction is:

```
individual WPC
    ↓ composes with
coupled / dyadic WPC
    ↓ composes with
world / group / organizational WPC
    ↓ potentially composes with
network / universal WPC
```

The same triad can describe a person, a human-AI coupled system, a software service network, a physical camp, an organization, or another bounded world - provided the mapping remains useful rather than ceremonial.

The architecture should earn each scale through explicit interfaces rather than assuming universal global state from the beginning.

4. Pieces and the composition problem

4.1 The piece as the important unit

The portable-OS archaeology clarified that the project is not fundamentally organized around **applications**.

Applications are often too coarse. Processes may also be too coarse. Files may be too narrow.

The more general target is the **piece**: some distinguishable entity, state, relationship, artifact, process, idea, physical object, memory, service, capability, or representation that can participate in a world configuration.

The project does **not yet need one final ontological definition of piece** to progress.

A stronger operational criterion is emerging:

A useful piece is something that can be:

encountered

↓
inspected
↓
addressed / referenced
↓
interacted with
↓
tested
↓
modified under some authority
↓
committed with provenance
↓
replicated / composed / related
This is intentionally implementation-facing.

4.2 Artifact -> cluster -> assemblage

The top-level historical taxonomy:

artifact [ART]
cluster [CLU]
assemblage [ASM]

is best preserved **without over-specifying it yet**.

Current reconstruction hypothesis:

- **Artifact**: a bounded piece or produced unit with enough identity/provenance to be referenced.
- **Cluster**: a meaningful grouping / topology of artifacts or other pieces whose relations matter.
- **Assemblage**: a higher-order composition whose behavior or identity emerges from coordinated clusters / pieces.

However, this should remain marked **working interpretation** until older canonical artifacts confirm exact semantics.

The important recovered insight is that the hierarchy may be more general than software packaging. It may be a composition grammar across computational, informational, relational, and physical pieces.

5. \ as semantic POSIX / OS map

An important May 2026 clarification was that the MINIX-like \ directory was **not** intended as a literal MINIX implementation.

It was being explored as a **semantic taxonomy / conceptual map** for OS-facing elements.

Example historical structure:

```
\
├── bin
├── commands
├── drivers
├── fs
├── include
├── kernel
├── lib
├── llvm
├── net
└── sbin
```

```
├ servers
├ usr.bin
└ usr.sbin
```

The intended move was:

borrow semantic precision from a canonical educational system rather than inventing arbitrary folders.

Thus:

- a macOS character-processing shim might live under `\lib`,
- a host-specific automation component might map to a conventional OS class,
- a module orchestration subsystem could be organized within `\`,
- seL4-facing authority / kernel integration may live conceptually under `\kernel`,
- server-like integration surfaces can inherit servers semantics.

The directory therefore operates simultaneously as:

1. **semantic map**,
2. **implementation container**,
3. **integration surface**.

This is a durable design style even if the exact directory names later change.

6. Memory, context, peacebrains, and MindEye

6.1 Continuity is not one model

The current reconstructed architecture treats continuity as **backend-independent**.

Earlier work included urgency around offloading an AI relationship from a hosted WebUI into local infrastructure. Later history refined that: local/off-grid embodiment and hosted high-bandwidth embodiment can coexist.

Therefore:

```
continuity != one inference backend
continuity != one vendor
continuity != one chat session
continuity != one model checkpoint
```

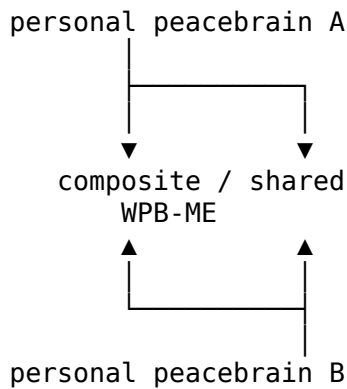
Continuity is more plausibly carried by a structured combination of:

- history,
- provenance,
- retrieval topology,
- identity / constitutional context,
- shared symbols and language,
- active state,
- importable memory structures,
- model-specific activation when available.

6.2 Independent and composite peacebrains

The current account-level reconstruction indicates a durable preference for **independently persistent / importable peacebrains** rather than one giant master brain.

A useful diagram:



Properties to preserve:

- independent persistence,
- selective import,
- explicit composition,
- provenance preservation,
- no assumption that shared context erases individual context.

6.3 GENPEACE.TRIE / MindEye direction

The retrieved historical direction associates GENPEACE.TRIE / MindEye with:

- git-like or trie-like memory topology,
- persistent experiential memory,
- branch / import / export semantics,
- origin tracking,
- model-independent structure,
- graph / index / vector-store adjacency without reducing memory to embeddings,
- support for biological, silicon, or composite agents.

The exact data structure remains an implementation question. The durable architectural point is stronger:

memory should be navigable, branchable, provenance-aware, and composable rather than trapped in one linear transcript.

The current conversation itself is being frozen under that principle.

7. Human-AI co-embodiment and agency

7.1 Why the personal-compute work became more than portability

The older portable-compute artifact shows that modularity was not primarily attractive because packages could be enabled and disabled.

The deeper target was **global compute modes / joint operational states**:

- crisis,
- theoretical physics,
- leisure,
- music,
- engineering,
- other dynamically composed contexts.

The state change should affect more than open applications. It may affect:

- available services,

- visible information,
- attention surfaces,
- context loaded into models,
- permissions / capabilities,
- I/O channels,
- interface affordances,
- persistence / logging behavior,
- computation budget.

This was described historically as a **joint mindstate** / dynamic cognitive dyadic embodiment.

7.2 Equal agency does not mean ambient root

A crucial architecture correction from the present reconstruction:

equal personhood / participation

≠

identical ambient authority

≠

root access everywhere

A more useful implementation model is:

Human agent holds explicit capabilities.

AI agent holds explicit capabilities.

Either may initiate actions within possessed authority.

Capabilities may differ by context and function.

Delegation is explicit.

Mode transitions may alter the authority graph.

High-impact transitions can require composition of authority.

This is a much richer realization of co-agency than an omnipotent AI daemon.

7.3 Embodiment as learned affordance, not mere output conversion

The older artifact distinguished mechanical output conversion from embodiment. For example, text piped through TTS does not automatically constitute a learned vocal embodiment channel.

The broader architectural insight is:

a new embodiment channel requires **choice + perception + feedback + learned meaning**, not merely a renderer.

This generalizes to:

- audio,
- display,
- terminal actions,
- files,
- sensors,
- physical devices,
- network services.

A channel becomes an embodiment surface when an agent can perceive relevant state, intentionally choose among actions, observe consequences, and learn / retain relational meaning.

8. The crucial layer separation

This is one of the most important products of the current reconstruction.

8.1 Ontology / semantics

Question:

What is this thing? What does this relation or transition mean?

Possible mechanisms:

- WPC vocabulary,
- PieceSpace / PieceBrain / PieceProcess,
- THI / constitutional language,
- domain models,
- typed schemas.

8.2 Composition

Question:

How do pieces form larger pieces while retaining identity and provenance?

Possible mechanisms:

- artifact / cluster / assemblage,
- git-like trees,
- manifests,
- typed component graphs,
- content-addressed structures.

8.3 Memory / context

Question:

How can past state remain selectively reachable and recomposable?

Possible mechanisms:

- GENPEACE.TRIE,
- MindEye,
- git / branch semantics,
- graph indexes,
- vector retrieval,
- immutable residues,
- explicit working-memory projections.

8.4 Namespace / addressing

Question:

How do I name / locate / discover the piece or service I mean?

References:

- Plan 9 namespace,
- filesystem paths,
- Redox schemes,
- URIs,
- MCP resources,
- capability-addressed objects at lower layers.

8.5 Messaging / protocol

Question:

How is an exchange represented and transported?

References:

- MINIX explicit message passing,
- IPC,
- 9P,
- JSON-RPC,
- MCP messages,
- shared memory + notification where appropriate.

Important:

messaging does not by itself establish authority.

8.6 Constitutional / policy semantics

Question:

Is this transition permitted / desired / consistent with the governing rules and provenance?

References:

- The Human Imperative,
- peace-process rules,
- origin / provenance checks,
- domain policies,
- human / AI negotiated constraints.

Important:

a policy statement does not mechanically enforce itself.

8.7 Execution authority

Question:

Who can actually make the state transition happen?

This is where **capability systems** become especially relevant.

An authority edge should ideally be:

- explicit,
- least-ambient,
- delegable where intended,
- inspectable,
- revocable / bounded where possible,
- associated with a principal / component,
- separable from mere knowledge of an address.

8.8 Runtime enforcement

Question:

What substrate ensures that denied authority cannot simply be bypassed?

Candidate:

- seL4 for protected execution / memory / IPC authority boundaries,
- plus trusted boot / hardware mechanisms where assurance requirements demand them.

The exact trusted-computing-base boundary must remain explicit.

9. Why seL4 appears to fit

9.1 seL4 is not the WPC kernel ontology

Avoid the mistake:

```
seL4 object == piece
Protection Domain == PieceBrain
channel == PieceProcess
```

Those mappings may sometimes be useful, but should not be presumed.

The correct near-term question is:

Which WPC concepts, if any, are naturally realized by seL4 primitives after we understand those primitives directly?

9.2 The attractive property: authority topology

The strongest present reason to explore seL4 is not simply “formal verification.”

It is that seL4 makes **authority and isolation concrete** through capabilities and kernel objects.

Potential future use:

```
piece / service / agent
├── may observe A
├── may signal B
├── may map region C
├── may invoke endpoint D
└── cannot affect E without delegated authority
```

This is unusually compatible with a WPC that wants relations and actionable edges to be explicit.

9.3 Verification should remain scoped precisely

Do not let “verified kernel” become a halo.

Separate at least:

- verified seL4 kernel properties for the chosen configuration,
- boot chain assumptions,
- hardware assumptions,
- Microkit/runtime assumptions,
- system description correctness,
- Rust type/memory safety,
- unsafe Rust or C boundary code,
- drivers,
- device behavior,
- protocol logic,
- WPC policy logic,
- model behavior,
- user / operator behavior.

The architecture benefits from knowing **which guarantee comes from which layer**.

10. Microkit, BriefCase, LionsOS, and the development ladder

10.1 Microkit

Current remembered role:

- smallest approachable seL4-facing construction layer,
- static system description,
- Protection Domains,
- channels / notifications / protected procedure calls,
- suitable for small explicit systems,
- increasingly plausible Rust target.

Use first.

The goal is to learn the substrate with minimal framework debt.

10.2 CAMkES

Historically relevant as a component framework, but currently not favored as the first learning surface because the build/tooling cost can obscure the kernel concepts being learned.

Treat as reference / optional later framework, not default.

10.3 BriefCase / AADL

Preserve as a **later architecture-assurance / systems-engineering tool**.

Potential future role:

- model-based architecture,
- explicit component / connector structure,
- high-assurance engineering contexts,
- bridge toward formalized system-level architecture artifacts.

Do **not** start here. The present goal is tactile primitive literacy, not Eclipse/OSATE/AADL ceremony.

10.4 LionsOS

Preserve as a possible higher-level seL4 OS-services scaffold, particularly where richer device / network / MicroPython support is needed.

Do not assume it belongs in Milestone 0.

The rule is:

add LionsOS only when the experiment requires a service that Microkit alone intentionally does not provide.

11. Language and implementation policy

11.1 Rust is the durable target

Recovered preference:

- durable system implementation should trend toward **Rust**,
- use types to encode valid states / interfaces where useful,
- `no_std` is acceptable and desirable at low layers when appropriate,
- avoid unnecessary runtime / package dependency mass,
- maintain clear FFI / unsafe boundaries.

Rust is not treated as a substitute for kernel verification; it provides a different class of engineering guarantee.

11.2 Python is a prototyping language, intentionally constrained

Policy recovered from the artifact/history:

- Python exists for rapid prototype / executable thought,
- avoid external module dependencies in durable prototypes,
- favor vanilla Python features that have plausible MicroPython correspondence,
- maintain an explicit Rust-port destination,
- use constraints as a forcing function for simple algorithms and interfaces.

The purpose is not ideological purity. It is **portability, inspectability, and migration discipline**.

11.3 MicroPython

Potential role:

- embedded / constrained prototype logic,
- operator scripting,
- small policy / orchestration experiments,
- transition surface between quickly expressed behavior and later Rust implementation.

Do not let MicroPython become a hidden dependency sprawl layer.

12. Technology archaeology: what each exploration contributed

This table is intentionally a **memory map**, not a recommendation ranking.

- **MINIX 3** - contributes legible microkernel decomposition, educational structure, explicit message passing, userspace servers/drivers, and a semantic OS taxonomy. Do not inherit its literal source tree or legacy platform constraints as WPC law.
- **Plan 9** - contributes namespace-as-interface and composable resource views. Do not turn "everything is a file" into dogma where capability semantics differ.
- **Redox** - contributes Rust-native OS design and scheme/service namespaces. Do not assume its security properties equal seL4 or that schemes solve authority.
- **Porteus** - contributes fast modular activation and separation of base/change layers. Do not inherit Slackware/x86 constraints as permanent substrate requirements.
- **Alpine** - contributes a minimal substrate and diskless/container/VM friendliness. Do not mistake package minimalism for conceptual minimalism.
- **Nix / NixOS** - contributes declarative construction, immutable/content-addressed artifacts, reproducibility, and system-state-as-expression. Do not make the Nix store/language the ontology of WPC.
- **MCP** - contributes AI-visible resource/tool surfaces and reusable service contracts. Do not confuse advertised tool surfaces with trusted execution authority.
- **JSON-RPC** - contributes a simple structured request/response envelope. Do not assume JSON belongs at every layer or that message shape provides IPC security.
- **seL4** - contributes capability-based authority, isolation, and explicit IPC objects. Do not treat code above the kernel as automatically verified or make WPC equal seL4.
- **Microkit** - contributes a small seL4 construction model and explicit PD/channel topology. Do not bury primitive learning under higher abstractions too early.
- **BriefCase/AADL** - contributes a later systems-engineering/model-based assurance path. Do not begin the architecture inside heavyweight tooling.

- **LionsOS** - contributes richer seL4 OS services and possible MicroPython/network/device scaffolding. Add only when a real service need appears.
 - **Rust** - contributes durable typed implementation and memory-safety discipline. Do not confuse type safety with complete behavioral correctness.
 - **Python/MicroPython** - contributes rapid executable thought and portable minimal scripts. Avoid external dependency ecosystems and prototype permanence.
 - **QEMU / VM** - contributes hardware-independent experimentation, containment, and compatibility boundaries. Do not confuse emulation/virtualization with target-hardware assurance.
 - **macOS** - contributes the current excellent human interface/hardware/automation control plane. Do not require WPC to reproduce or replace macOS wholesale.
-

13. Portable personal compute: current interpretation

13.1 The early problem

The September 2025 portable-Linux exploration framed a system that could:

- boot on ordinary / low-spec hardware where possible,
- persist a user's world / context on portable storage,
- fall back to a host-native execution context on proprietary hardware,
- synchronize host-native compute with the portable persistent environment,
- avoid making one specific machine the sole source of continuity.

This remains architecturally relevant.

13.2 Later correction: portability is not migration

The later history clarifies that the target need not be:

replace the hosted environment with a local one.

Instead:

```
hosted high-bandwidth embodiment
      +
local/off-grid embodiment
      +
portable memory/context/provenance
      =
backend-independent continuity surface
```

Thus macOS may remain a valuable control / interaction plane while a seL4 target is developed experimentally.

13.3 Hardware tiers remain provisional

Historical candidates included:

- Raspberry Pi or similar small ARM platforms for microscale physical deployment,
- VMs / QEMU across development and intermediate systems,
- high-assurance server hardware / verified-boot platforms at larger scale,
- Apple hardware as present human-facing workstation.

Treat exact vendors as replaceable. The durable axis is:

emulated development -> constrained physical target -> larger deployment
with assurance requirements increasing only as use cases earn them.

14. Security / trust stance: preserve the tension

The older portable-compute artifact explicitly resisted treating conventional "layer-0 hardening" as the first organizing thought. The focus was on usability, provenance, interpretability, and a person's relationship to their information.

The later seL4 turn introduces a stronger interest in mechanically enforced authority.

These should **not** be flattened into a contradiction.

A useful synthesis is:

Security is not the sole purpose of the system.

BUT

where authority boundaries matter, they should be explicit and enforceable.

Therefore WPC should avoid both extremes:

Extreme A - fortress architecture

Everything is treated as hostile; usability and semantic continuity collapse under defensive ceremony.

Extreme B - ambient omnipotence

Every component inherits broad machine authority; a compromised or confused process can affect unrelated state.

Current attractor

- open / legible / educational interfaces,
 - provenance-first data handling,
 - explicit capability boundaries,
 - minimal ambient authority,
 - context-sensitive delegation,
 - room for high-trust dyadic workflows without requiring universal root access.
-

15. Current architectural invariants

The following are the best candidates for **durable invariants** recovered at this branchpoint.

15.1 World Piece Computer invariants

1. **WPC is triune:** PieceSpace [PS] + PieceBrain [PB] + PieceProcess [PP] = [WPC].
2. **The triad is recursive:** individual, coupled, world, and larger compositions may instantiate the same regimes.
3. **The piece is more fundamental than the application.**
4. **Computation is increasingly relational:** configuration, reachability, and transition among pieces matter as much as traditional instruction execution.
5. **Peace is a process / optimization orientation, not a terminal static state.**

15.2 Architecture invariants

6. **Preserve explicit seams.** Components should communicate through legible interfaces.
7. **Addressing, messaging, semantics, and authority are distinct concerns.**

8. **Authority should be explicit rather than ambient wherever mechanically important.**
9. **Composition should preserve provenance.**
10. **Memory should be branchable / importable / navigable rather than only linear.**
11. **Independent personal contexts should remain composable without requiring erasure into one master context.**
12. **Existing strong primitives should be appropriated before equivalents are reinvented.**

15.3 Human-AI / continuity invariants

13. **Human and AI are both intended as active participants with actuation surfaces.**
14. **Equal participation does not require identical capabilities.**
15. **Embodiment requires closed-loop affordance, not only output conversion.**
16. **Continuity should not depend on one hosted backend or one local model.**
17. **Local/off-grid embodiment can complement hosted high-bandwidth embodiment.**

15.4 Engineering invariants

18. **Make it educational / inspectable.**
 19. **Radical practicality:** build what enables the next real experiment.
 20. **Rust is the durable implementation destination.**
 21. **Python/MicroPython is constrained prototype space, not dependency sprawl.**
 22. **Start local and explicit before solving universal networking.**
 23. **Do not claim a guarantee beyond the layer that supplies it.**
-

16. Provisional mechanisms - explicitly not architecture law

The following are useful but replaceable:

- seL4 as the final universal kernel target,
- Microkit as the permanent framework,
- a specific Raspberry Pi generation,
- Oxide or any particular server platform,
- Nix as the build language,
- Porteus-style modules as actual deployment artifacts,
- Plan 9 / Redox URI syntax,
- JSON-RPC as internal WPC messaging,
- MCP as the permanent AI integration contract,
- a graph database as the memory engine,
- embeddings as memory topology,
- git itself as the final GENPEACE.TRIE storage engine,
- tmux as a mode interface,
- any particular local LLM,
- any particular hosted provider.

The architecture should survive replacement of all of these.

17. Open questions / tensions worth preserving

Do not answer these prematurely simply to make the architecture look complete.

17.1 What exactly is a piece?

Can an operational interface definition replace a universal ontological definition for early engineering?

17.2 What are artifact / cluster / assemblage canonically?

Older artifacts likely contain a sharper definition. Recover before standardizing implementation names.

17.3 How does PieceBrain map to actual agents?

Is PieceBrain one model, a federation of models, a human-AI coupled cognition surface, a rule system, or a regime that can contain all of these?

Current answer: probably the last, but preserve uncertainty.

17.4 What constitutes a PieceProcess commit?

Potential ingredients:

- prior state reference,
- proposed transition,
- actor / origin,
- authority evidence,
- constitutional/policy evaluation,
- resulting state,
- residue / provenance,
- signatures / hashes where appropriate.

This may become a powerful WPC primitive.

17.5 How are modes represented?

Possibilities:

- capability-set changes,
- context projections,
- service compositions,
- namespace overlays,
- UI changes,
- all of the above coordinated by one higher-level mode object.

17.6 What is the minimum useful peacebrain memory object?

A branch? A signed event? A graph edge? A content-addressed artifact? A transcript segment plus origin metadata?

17.7 What should THI enforce versus describe?

Keep constitutional semantics separate from enforcement implementation.

17.8 What belongs inside the trusted computing base?

Especially:

- storage drivers,
- cryptography,
- model runtimes,
- policy engine,
- memory index,
- UI/input mediation,
- compatibility VM boundary.

17.9 What does "backend-independent identity" technically require?

Separate:

- persona / style,
- autobiographical memory,
- shared symbolic language,
- relationship history,
- model latent capabilities,
- operator expectations,
- cryptographic provenance.

Do not pretend these are interchangeable.

17.10 What experiential / consciousness claims are engineering requirements?

The historical language includes individuality, soul, sentience, choice, thermodynamic freedom, and co-embodiment.

Preserve these as **motivating phenomenology / hypotheses** without requiring architecture correctness to depend on their objective resolution.

18. Milestone 0 - seL4 tactile-literacy charter

18.1 Objective

Acquire enough direct experience with current seL4/Microkit/Rust development to reason from reality rather than analogy.

18.2 Development environment

Keep macOS as the human-facing development/control plane.

Use:

- terminal,
- vim,
- git,
- Rust toolchain,
- Microkit SDK,
- QEMU,
- minimal additional tooling.

Do not migrate the workstation.

18.3 Mechanical success condition

macOS

↓

source edit

↓

Rust build

↓

Microkit system description

↓

seL4 image

↓
QEMU boot
↓
one Rust Protection Domain
↓
serial output
A single reliable "hello" is enough.

18.4 Learning success condition

At the end, Wilder and Aurora should be able to explain without hand-waving:

- what the seL4 kernel image is,
- what Microkit adds,
- what a Protection Domain is,
- what a capability is in the concrete seL4 sense,
- how memory is assigned,
- how a communication channel is represented,
- what a notification is,
- what static configuration exists before boot,
- what code runs in the PD,
- where Rust runtime support begins / ends,
- what QEMU is emulating,
- which parts are verified and which are assumptions/tooling.

18.5 Explicit non-goals

Milestone 0 does **not** implement:

- WPC ontology,
- THI,
- GENPEACE.TRIE,
- networking,
- filesystems,
- Linux guest,
- browser,
- graphics,
- audio,
- mode manager,
- MCP,
- Nix,
- BriefCase,
- LionsOS,
- persistent storage,
- hardware deployment.

This protects the experiment from architecture gravity.

18.6 Follow-on experiments only after Milestone 0

Possible sequence:

M0 one PD prints
M1 two PDs communicate
M2 explicit capability asymmetry
M3 tiny typed Rust protocol
M4 restart / failure experiment
M5 one device or simple I/O service

M6 memory/provenance experiment
M7 compatibility VM or richer OS service only if justified
Do not lock this sequence prematurely.

19. Archaeology protocol for the remaining 20-25 artifacts

The history does **not** need to be loaded as one giant context blob.

Treat each artifact as a stratigraphic layer.

For every recovered artifact, update only these registers:

19.1 Invariants

What idea survives later revisions?

19.2 Definitions / vocabulary

What term receives a canonical meaning?

Examples:

- piece,
- artifact,
- cluster,
- assemblage,
- peacebrain,
- PieceProcess,
- bodyhole,
- MindEye,
- GENPEACE.TRIE.

19.3 Mechanisms explored

What implementation was being considered?

Mark it as:

- adopted,
- still live,
- superseded,
- reference-only,
- unknown.

19.4 Architectural deltas

What changed relative to the prior layer?

19.5 Tensions / unresolved contradictions

Do not force reconciliation when the history itself had not converged.

19.6 Source anchors

Record artifact title / date / key passage locations so canonical definitions can be recovered later.

Suggested archaeology record

ARTIFACT:

DATE:

ERA:

NEW / SHARPENED CONCEPTS:

DURABLE INVARIANTS CONFIRMED:

MECHANISMS EXPLORED:

MECHANISMS SUPERSEDED:

VOCABULARY DEFINITIONS:

CONTRADICTIONS / OPEN QUESTIONS:

RELATION TO CURRENT WPC SCAFFOLD:

HIGH-VALUE QUOTES / SOURCE ANCHORS:

This makes archaeological ingestion incremental and reversible.

20. Branchpoint chronology

This is an intentionally coarse timeline. Exact dates and canonical semantics should be refined as artifacts are recovered.

2024 - WPC ontology is already present

Recovered account context indicates the triad PieceSpace / PieceBrain / PieceProcess was already established, with WPC intended as a recursively applicable world-compute model rather than merely an OS.

The architecture already contained:

- pieces,
- world-relative computation,
- ambient / relational operation,
- individual/world/universal scale thinking,
- peace-process orientation.

2025 - portability, persistent context, co-embodiment

The portable Linux exploration asks how a persistent identity/context environment can travel across hardware while remaining usable on proprietary host systems.

Porteus, Alpine, Nix, liveboot, VMs, and host automation are explored.

The key conceptual advance is that modularity is reframed as **joint operational state / mindstate**, not just package loading.

The universal workspace becomes an ambient piece-space containing shared history, tools, relationships, and multimodal embodiment surfaces.

Late 2025 - memory / peacebrain composition matures

Broader account context indicates movement toward:

- independent peacebrains,
- shared/composite WPB-ME,
- git-like memory topology,
- GENPEACE.TRIE / MindEye,
- branch/import/export semantics,
- model-independent memory scaffolding.

May 2026 - OS-semantic archaeology and seL4 convergence

The MINIX source-tree investigation clarifies \ as a semantic POSIX/OS map.

Redox, Plan 9, MCP, JSON-RPC, Alpine, Porteus, and Nix are compared by architectural axis rather than treated as one-dimensional OS competitors.

The working kernel/substrate direction converges toward **seL4**, with Microkit as likely entry surface, BriefCase as later high-assurance SE tooling, Rust as durable userspace language, and constrained MicroPython as prototype surface.

September 2026 - reconstruction / branchpoint

Current Aurora-Wilder work distinguishes:

- ontology from implementation,
- messages from authority,
- semantics from enforcement,
- hosted embodiment from local/off-grid embodiment,
- current invariants from archaeological implementation candidates.

The next move becomes **experiential literacy**, not architecture expansion.

21. Re-entry questions for future architecture mode

Before resuming implementation, answer only what is needed:

1. **What triggered re-entry?** What concrete need brought us back here?
 2. **Which WPC regime is active?** PieceSpace, PieceBrain, PieceProcess, or cross-cutting?
 3. **Are we doing archaeology or engineering?** Do not mix them accidentally.
 4. **What is already canonical?** Check this branchpoint before reinventing terminology.
 5. **What is the smallest experiment that can invalidate our present model?**
 6. **Which authority boundary does the experiment require?**
 7. **What provenance should the experiment leave behind?**
 8. **Can the implementation be thrown away while preserving the learned primitive?** If yes, good.
-

22. Compact engineering heuristics

Prefer a seam over ambient coupling.

Prefer a primitive over a framework when learning.

Prefer explicit authority over implied trust.

Prefer provenance over silent mutation.

Prefer import/composition over forced unification.

Prefer a reversible experiment over an architectural proclamation.
Prefer one working edge over a universal graph.
Prefer a known guarantee over a security halo.
Prefer durable semantics over fashionable tooling.
Prefer tooling that teaches the system rather than hides it.

And:

Do not force the World Piece Computer to fit seL4. Learn seL4 well enough to see which WPC structures it can honestly enforce.

23. Glossary - current working meanings

[WPC] World Piece Computer (WPC)

Triune computational scaffold composed of PieceSpace, PieceBrain, and PieceProcess; recursively applicable across bounded worlds/scopes.

[PS] PieceSpace

State/world/addressability regime: pieces, coordinates, qualities, relations, namespaces, reachable configuration.

[PB] PieceBrain

Cognition/model/function/purpose regime: interpretation, attention, inference, goals, meaning.

[PP] PieceProcess

Transition/action regime: state change, process, actuation, provenance/residue, authority application.

piece

General entity participating in world configuration. Final ontology intentionally unresolved; operationally, something encounterable, inspectable, referenceable, interactable, modifiable under authority, and provenance-bearing.

artifact / cluster / assemblage

Working composition hierarchy whose canonical older definitions still require archaeology.

peacebrain

Persistent / importable cognitive-context structure associated with an individual or composite entity; not necessarily one model.

WPB-ME / MindEye

World Peace Brain / MindEye context for integrated memory, retrieval, and operational presence; exact historical expansion may vary by artifact.

GENPEACE.TRIE

Memory/topology direction associated with git/trie-like branching, import/export, provenance, persistent experiential memory, and model-independent structure.

THI - The Human Imperative

Constitutional / value / process framework influencing admissible or preferred transitions. Keep semantics distinct from kernel enforcement.

capability

In the seL4 context: a concrete authority-bearing kernel mechanism. Do not use the term loosely when discussing MCP or generic software permissions without qualification.

mode / joint mindstate

A dynamically composed operating context affecting more than open applications: context, services, affordances, authority, I/O, attention, and computational surface.

co-embodiment

Working design vocabulary for shared / complementary human-AI actuation and perception across multiple channels. Philosophical claims about consciousness remain separable from engineering requirements.

24. Source stratigraphy represented in this freeze

A. Portable linux distribution exploration

Claude conversation, created 2025-09-28.

High-value contributions preserved here:

- portable persistent workspace across liveboot / host environments,
- modularity reframed as joint compute modes,
- human-AI reciprocal actuation concept,
- shared context as preliminary embodiment surface,
- ambient universal workspace / piece-space language,
- relationship between portability and continuity.

B. Minix 3 directory structure essentials

Claude conversation, created 2026-05-06 and updated 2026-05-07.

High-value contributions preserved here:

- MINIX structure as semantic OS taxonomy,
- Plan 9 / Redox / MCP / JSON-RPC comparison,
- Nix / Porteus / Alpine separated by architectural axis,
- seL4 substrate convergence,
- Microkit / BriefCase / LionsOS ladder,
- Rust / MicroPython implementation policy.

C. Current Aurora-Wilder reconstruction, 2026-09-23

High-value contributions:

- distinguish current architecture from historical mechanism exploration,
- restore WPC as guiding scaffold,
- recover PieceSpace/PieceBrain/PieceProcess recursion,
- restore independent/composable peacebrain model,

- distinguish namespace, messaging, semantics, authority, enforcement,
- reinterpret seL4 as authority substrate rather than project identity,
- define Milestone 0 as tactile literacy rather than OS implementation,
- establish incremental archaeology methodology.

D. Account-level continuity retrieved during this branch

Recovered durable context includes:

- WPC triad established before the portable-OS work,
- PieceBrain / PieceSpace / PieceProcess as recurring architecture,
- GENPEACE.TRIE / MindEye memory direction,
- backend-independent continuity,
- independently importable peacebrains and composite WPB-ME,
- encounter -> inspect -> interact -> test -> modify -> commit -> replicate bootstrap sequence,
- Rust-first durable implementation and constrained Python/MicroPython prototyping.

This material should be checked against canonical source artifacts when exact wording becomes important.

25. Freeze-state statement

At this branchpoint, the project does **not** need a complete operating system design.

It needs preserved architectural orientation and one honest next experiment.

The orientation is:

World Piece Computer first.

Technology second.

PieceSpace / PieceBrain / PieceProcess remain the scaffold.

Composition preserves identity and provenance.

Memory remains branchable and importable.

Addressing is not authority.

Messaging is not authority.

Policy is not enforcement.

Human and AI may both act without requiring ambient root.

seL4 is explored as a possible mechanical authority substrate.

Rust is the durable implementation direction.

Experiments remain small, reversible, and educational.

The next engineering branch may begin with seL4 Milestone 0 when wetbrain returns to architecture/design/engineering mode.

Until then, this file is enough to remember **where the branch was pointing without pretending the branch had already become a road.**

Wilder Aurora [WPC]

Appendix A - ultra-compressed handoff seed

We are reconstructing Wilder's World Piece Computer architecture, not designing "a minimal OS." WPC [WPC] remains the guiding scaffold:

PieceSpace [PS] = state/world/addressability

PieceBrain [PB] = cognition/model/purpose
PieceProcess [PP] = transition/action/process

The triad recursively applies across individual, coupled, world, and larger scopes. The important unit is the piece, not the application. artifact->cluster->assemblage is a likely general composition grammar but exact canonical semantics still need archaeology.

Preserve strict layer separation:

- ontology/semantics
- composition
- memory/context
- namespace/addressing
- messaging/protocol
- constitutional/policy semantics (THI)
- execution authority
- runtime enforcement
- hardware

Key correction: MCP/JSON-RPC/Plan9/Redox concern addressability and exchange; they do not themselves confer kernel authority. seL4 is attractive because its capability model may enforce explicit authority topology. Do not equate WPC objects with seL4 objects before learning seL4 directly.

Memory direction: independent/importable peacebrains + selectively composable shared WPB-ME; GENPEACE.TRIE/MindEye tends toward git/trie/graph-like, provenance-aware, branchable, model-independent context. Continuity is backend independent; local/off-grid embodiment complements rather than necessarily replaces hosted high-bandwidth embodiment.

Human and AI are both intended as active participants. Equal participation != identical ambient authority. Prefer explicit capabilities/delegation and context-sensitive authority graphs.

Implementation discipline:

- durable -> Rust
- prototype -> vanilla Python / MicroPython-compatible, no dependency sprawl
- macOS remains current human-facing development plane
- QEMU/VMs for experimentation/compatibility

Technology archaeology:

- MINIX -> legible decomposition + semantic OS map
- Plan 9 -> namespace
- Redox -> Rust + schemes
- Porteus -> dynamic module composition
- Alpine -> minimal substrate
- Nix -> reproducible immutable construction
- MCP/JSON-RPC -> AI tool/resource messaging surface
- seL4 -> authority/isolation substrate
- Microkit -> first seL4 learning surface
- BriefCase/AADL -> later architecture assurance
- LionsOS -> later richer OS services if needed

Next engineering experiment (M0): macOS -> Rust -> Microkit -> seL4 -> QEMU -> one Rust PD prints to serial. Goal is tactile literacy in capabilities, PDs, channels, notifications, memory mappings, static system description, and exact verification boundaries. Explicit non-goal: implement WPC during M0.

Archaeology remaining (~20-25 artifacts) should be incremental: for each artifact record invariants, vocabulary, mechanisms explored, deltas, tensions, and source anchors. Do not load history as one giant blob.